Практическая работа №22 Тема: Циклические алгоритмы

Цель лабораторной работы: изучить простейшие средства отладки программ в среде Visual Studio. Составить и отладить программу циклического алгоритма.

4.1. Операторы организации циклов

Под *циклом* понимается многократное выполнение одних и тех же операторов при различных значениях промежуточных данных. Число повторений может быть задано в явной или неявной форме.

К операторам цикла относятся: *цикл с предусловием* `while`, *цикл с постусловием* `do while`, *цикл с параметром* `for` и *цикл перебора* `foreach`. Рассмотрим некоторые из них.

4.2. Цикл с предусловием

Оператор цикла `while` организует выполнение одного оператора (простого или составного) неизвестное заранее число раз. Формат цикла `while`:

```
while (B) S;
```

где `B` – выражение, истинность которого проверяется (условие завершения цикла); `S` – тело цикла – оператор (простой или составной).

Перед каждым выполнением тела цикла анализируется значение выражения `B`: если оно истинно, то выполняется тело цикла, и управление передается на повторную проверку условия `B`; если значение `B` ложно – цикл завершается и управление передается на оператор, следующий за оператором `S`.

Если результат выражения `B` окажется ложным при первой проверке, то тело цикла не выполнится ни разу. Отметим, что если условие `B` во время работы цикла не будет изменяться, то возможна ситуация зацикливания, то есть невозможность выхода из цикла. Поэтому внутри тела должны находиться операторы, приводящие к изменению значения выражения `B` так, чтобы цикл мог корректно завершиться.

В качестве иллюстрации выполнения цикла `while` рассмотрим программу вывода целых чисел от 1 до n по нажатию кнопки на форме:

```
private void button1_Click(object sender, EventArgs e)
{
```

```
        int n = 10;  // Количество повторений цикла
        int i = 1;    // Начальное значение
        while (i <= n) // Пока i меньше или равно n
        {
            MessageBox.Show(i.ToString()); // Показываем i
            i++; // Увеличиваем i на 1
        }
    }
```

4.3. Цикл с постусловием

Оператор цикла do while также организует выполнение одного оператора (простого или составного) неизвестное заранее число раз. Однако в отличие от цикла while условие завершения цикла проверяется после выполнения тела цикла. Формат цикла do while:

```
do S while (B);
```

где B – выражение, истинность которого проверяется (условие завершения цикла); S – тело цикла – оператор (простой или блок).

Сначала выполняется оператор S, а затем анализируется значение выражения B: если оно истинно, то управление передается оператору S, если ложно – цикл завершается, и управление передается на оператор, следующий за условием B. Так как условие B проверяется после выполнения тела цикла, то в любом случае тело цикла выполнится хотя бы один раз.

В операторе do while, так же как и в операторе while, возможна ситуация зацикливания в случае, если условие B всегда будет оставаться истинным.

4.4. Цикл с параметром

Цикл с параметром имеет следующую структуру:

```
for (<инициализация>; <выражение>; <модификация>)
     <оператор>;
```

Инициализация используется для объявления и/или присвоения начальных значений величинам, используемым в цикле в качестве параметров (счетчиков). В этой части можно записать несколько операторов, разделенных запятой. Областью действия переменных, объявленных в части инициализации цикла, является цикл и вложенные блоки. Инициализация выполняется один раз в начале исполнения цикла.

Выражение определяет условие выполнения цикла: если его результат истинен, цикл выполняется. Истинность выражения проверяется перед каждым выполнением тела цикла, таким образом, цикл с пара-

метром реализован как *цикл с предусловием*. В блоке *выражение* через запятую можно записать несколько логических выражений, тогда запятая равносильна операции *логическое И* (&&).

Модификация выполняется после каждой итерации цикла и служит обычно для изменения параметров цикла. В части *модификация* можно записать несколько операторов через запятую.

Оператор (простой или составной) представляет собой тело цикла.

Любая из частей оператора `for` (инициализация, выражение, модификация, оператор) может отсутствовать, но точку с запятой, определяющую позицию пропускаемой части, надо оставить.

Пример формирования строки, состоящей из чисел от 0 до 9, разделенных пробелами:

```
string s = ""; // Инициализация строки
for (int i = 0; i <= 9; i++) // Перечисление всех чисел
   s += i.ToString() + " "; // Добавляем число и пробел
MessageBox.Show(s.ToString()); // Показываем результат
```

Данный пример работает следующим образом. Сначала вычисляется начальное значение переменной `i`. Затем, пока значение `i` меньше или равно 9, выполняется тело цикла, и затем повторно вычисляется значение `i`. Когда значение `i` становится больше 9, условие – ложно и управление передается за пределы цикла.

4.5. Средства отладки программ

Практически в каждой вновь написанной программе после запуска обнаруживаются ошибки.

Ошибки первого уровня (ошибки компиляции) связаны с неправильной записью операторов (орфографические, синтаксические). При обнаружении ошибок компилятор формирует список, который отображается по завершению компиляции (рис. 4.1). При этом возможен только запуск программы, которая была успешно скомпилирована для предыдущей версии программы.

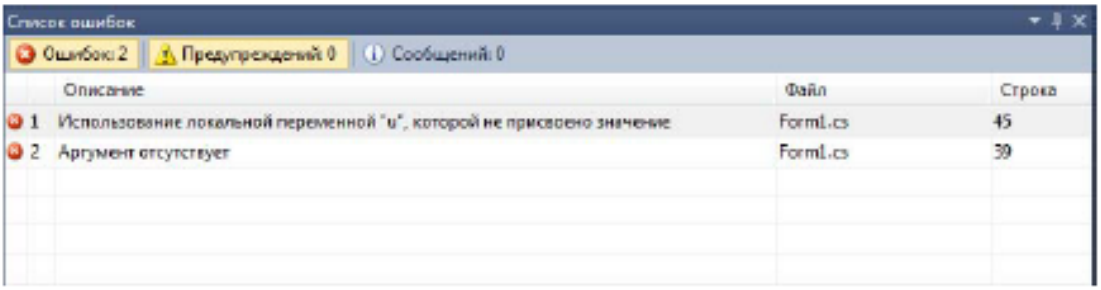

Рис. 4.1. Окно со списком ошибок компиляции

При выявлении ошибок компиляции в нижней части экрана появляется текстовое окно (см. рис. 4.1), содержащее сведения обо всех ошибках, найденных в проекте. Каждая строка этого окна содержит имя файла, в котором найдена ошибка, номер строки с ошибкой и характер ошибки. Для быстрого перехода к интересующей ошибке необходимо дважды щелкнуть на строке с ее описанием. Следует обратить внимание на то, что одна ошибка может повлечь за собой другие, которые исчезнут при ее исправлении. Поэтому необходимо исправлять их последовательно, сверху вниз и, после исправления каждой – компилировать программу снова.

Ошибки второго уровня (ошибки выполнения) связаны с ошибками выбранного алгоритма решения или с неправильной программной реализацией алгоритма. Эти ошибки проявляются в том, что результат расчета оказывается неверным либо происходит переполнение, деление на ноль и др. Поэтому перед использованием отлаженной программы ее надо протестировать, т. е. сделать просчеты при таких комбинациях исходных данных, для которых заранее известен результат. Если тестовые расчеты указывают на ошибку, то для ее поиска следует использовать встроенные средства отладки среды программирования.

В простейшем случае для локализации места ошибки рекомендуется поступать следующим образом. В окне редактирования текста установить точку останова перед подозрительным участком, которая позволит остановить выполнение программы и далее более детально следить за ходом работы операторов и изменением значений переменных. Для этого достаточно в окне редактирования кода щелкнуть слева от нужной строки. В результате чего данная строка будет выделена красным (рис. 4.2).

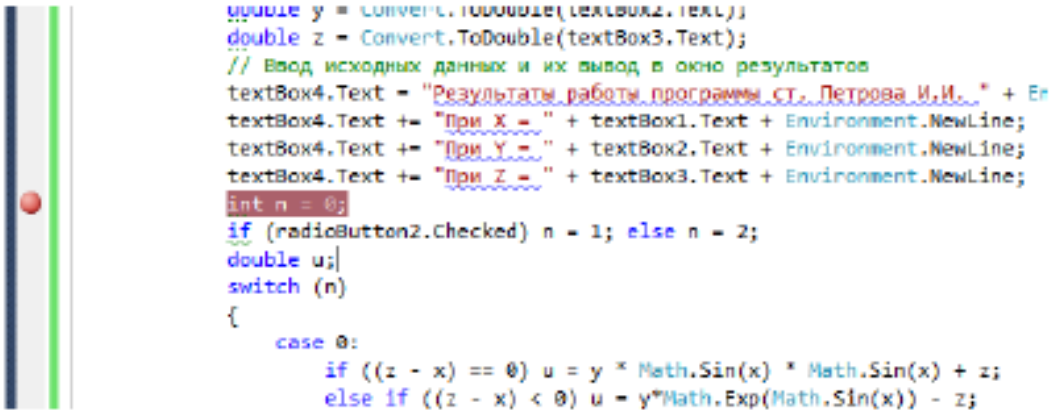

Рис. 4.2. Фрагмент кода с точкой останова

При выполнении программы и достижения установленной точки программа будет остановлена, и далее можно выполнять код по шагам с помощью команд *Отладка → Шаг с обходом* (без захода в методы) или *Отладка → Шаг с заходом* (с заходом в методы) (рис. 4.3).

```
textBox4.Text += "При Z = " + textBox3.Text + Environment.NewLine;
int n = 0;
if (radioButton2.Checked) n = 1;
else if (radioButton3.Checked) n = 2;
double u;
switch (n)
{
    case 0:
        if ((z - x) == 0) u = y * Math.Sin(x) * Math.Sin(x) + z;
        else if ((z - x) < 0) u = y*Math.Exp(Math.Sin(x)) - z;
        else u = y*Math.Sin(Math.Sin(x)) + z;
        textBox4.Text += "U = " + Convert.ToString(u) + Environment
```

Рис. 4.3. Отладка программы

Желтым цветом выделяется оператор, который будет выполнен. Значение переменных во время выполнения можно увидеть, наведя на них курсор. Для прекращения отладки и остановки программы нужно выполнить команду меню *Отладка → Остановить отладку*.

Для поиска алгоритмических ошибок можно контролировать значения промежуточных переменных на каждом шаге выполнения подозрительного кода и сравнивать их с результатами, полученными вручную.

4.6. Порядок выполнения задания

З а д а н и е : Вычислить и вывести на экран таблицу значений функции $y = a \cdot \ln(x)$ при x, изменяющемся от x_0 до x_k с шагом dx, a – константа.

Панель диалога представлена на рис. 4.4. Текст обработчика нажатия кнопки **Вычислить** приведен ниже.

```
private void button1_Click(object sender, EventArgs e)
{
    // Считывание начальных данных
    double x0 = Convert.ToDouble(textBox1.Text);
    double xk = Convert.ToDouble(textBox2.Text);
    double dx = Convert.ToDouble(textBox3.Text);
    double a = Convert.ToDouble(textBox4.Text);
    textBox5.Text = "Работу выполнил ст. Иванов М.А." +
        Environment.NewLine;
    // Цикл для табулирования функции
    double x = x0;
    while (x <= (xk + dx / 2))
    {
        double y = a * Math.Log(x);
        textBox5.Text += "x=" + Convert.ToString(x) +
                         "; y=" + Convert.ToString(y) +
                         Environment.NewLine;
        x = x + dx;
    }
}
```

После отладки программы следует проверить правильность работы программы с помощью контрольного примера (см. рис. 4.4). Установите точку останова на оператор перед циклом и запустите программу. После попадания на точку остановки, выполните пошагово программу и проследите, как меняются все переменные в процессе выполнения.

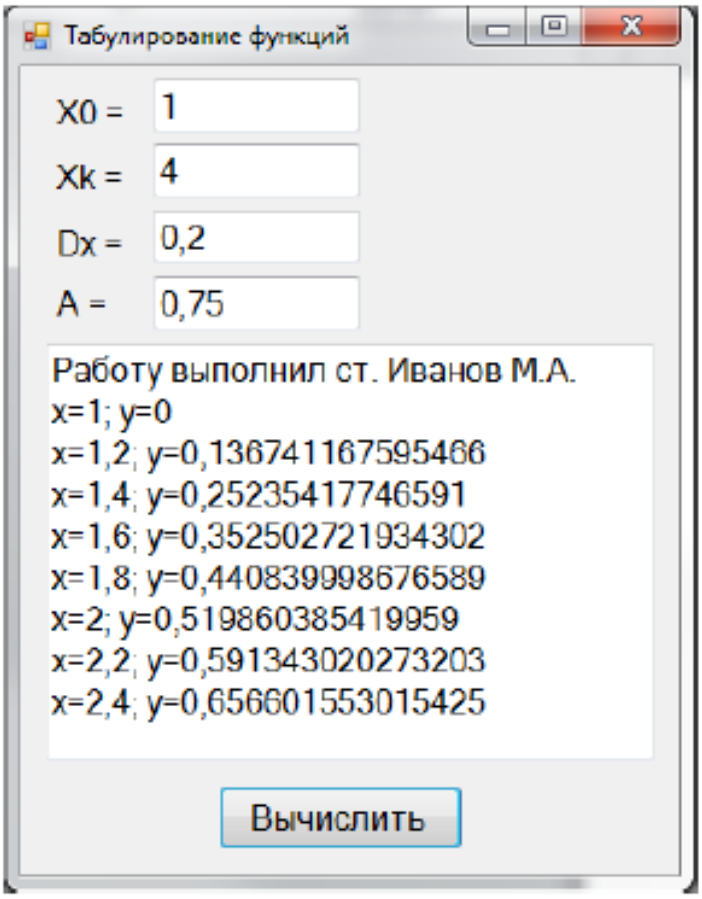

Рис. 4.4. Окно программы для табулирования функции

Индивидуальные задания

Составьте программу табулирования функции $y(x)$, выведите на экран значения x и $y(x)$. Нужный вариант задания выберите из нижеприведенного списка по указанию преподавателя. Откорректируйте элементы управления в форме в соответствии со своим вариантом задания.

1) $y = 10^{-2}bc / x + \cos\sqrt{a^3 x}$,
$x_0 = -1.5; x_k = 3.5; dx = 0.5;$
$a = -1.25; b = -1.5; c = 0.75;$

2) $y = 1.2(a\text{-}b)^3 e^{x^2} + x$,
$x_0 = -0.75; x_k = -1.5; dx = -0.05;$
$a = 1.5; b = 1.2;$

3) $y = 10^{-1} a x^3 tg(a - bx)$,
$x_0 = -0.5; x_k = 2.5; dx = 0.05;$
$a = 10.2; b = 1.25;$

4) $y = ax^3 + \cos^2(x^3 - b)$,
$x_0 = 5.3; x_k = 10.3; dx = 0.25;$
$a = 1.35; b = -6.25;$

5) $y = x^4 + \cos(2 + x^3 - d)$,
$x_0 = 4.6; x_k = 5.8; dx = 0.2;$
$d = 1.3;$

6) $y = x^2 + tg(5x + b/x)$,
$x_0 = -1.5; x_k = -2.5; dx = -0.5;$
$b = -0.8;$

7) $y = 9(x + 15\sqrt{x^3 + b^3})$,
$x_0 = -2.4; x_k = 1; dx = 0.2;$
$b = 2.5;$

8) $y = 9x^4 + \sin(57.2 + x)$,
$x_0 = -0.75; x_k = -2.05; dx = -0.2;$

9) $y = 0.0025bx^3 + \sqrt{x + e^{0.82}}$,
$x_0 = -1; x_k = 4; dx = 0.5;$
$b = 2.3;$

10) $y = x \cdot \sin(\sqrt{x + b - 0.0084})$,
$x_0 = -2.05; x_k = -3.05; dx = -0.2;$
$b = 3.4;$

11) $y = x + \sqrt{|x^3 + a - be^x|}$,
$x_0 = -4; x_k = -6.2; dx = -0.2;$
$a = 0.1;$

12) $y = 9(x^3 + b^3)tgx$,
$x_0 = 1; x_k = 2.2; dx = 0.2;$
$b = 3.2;$

13) $y = |x - b|^{1/2} / |b^3 - x^3|^{3/2} + \ln|x - b|$,
$x_0 = -0.73; x_k = -1.73; dx = -0.1;$
$b = -2;$

14) $y = (x^{5/2} - b)\ln(x^2 + 12.7)$,
$x_0 = 0.25; x_k = 5.2; dx = 0.3;$
$b = 0.8;$

15) $y = 10^{-3}|x|^{5/2} + \ln|x + b|$,
$x_0 = 1.75; x_k = -2.5; dx = -0.25;$
$b = 35.4;$

16) $y = 15.28|x|^{-3/2} + \cos(\ln|x| + b)$,
$x_0 = 1.23; x_k = -2.4; dx = -0.3;$
$b = 12.6;$

17) $y = 0.00084(\ln|x|^{5/4} + b)/(x^2 + 3.82)$,
$x_0 = -2.35; x_k = -2; dx = 0.05;$
$b = 74.2;$

18) $y = 0.8 \cdot 10^{-5}(x^3 + b^3)^{7/6}$,
$x_0 = -0.05; x_k = 0.15; dx = 0.01;$
$b = 6.74;$

19) $y = (\ln(\sin(x^3 + 0.0025)))^{3/2} + 0.8 \cdot 10^{-3}$,
$x_0 = 0.12; x_k = 0.64; dx = 0.2;$

20) $y = a + x^{2/3}\cos(x + e^x)$,
$x_0 = 5.62; x_k = 15.62; dx = 0.5;$
$a = 0.41$